Polyglotic LoRA - Channel-Aware, Language-Routed Low-Rank Adaptation (PLoRA)

Amitava Das[⋆]

[⋆]Pragya Lab, BITS Pilani Goa, India

Abstract

Polyglotic LoRA (PLoRA) is a ***channel-aware, language-routed*** **parameter-efficient fine-tuning (PEFT)** framework for multilingual large language models. While **LoRA-style low-rank adaptation** is an efficient route to specialization, multilingual deployments remain bottlenecked by ***negative transfer***: a *single shared* low-rank update is forced to serve languages whose internal representations diverge across depth, inducing **cross-language interference**, suppressing **minority-language gains**, and destabilizing behavior under domain shift and code-switching. **PLoRA** addresses this by explicitly modeling ***language-specific channels***—persistent, depth-dependent signatures in representation geometry—and using them to **(i)** ***place*** **adapters sparsely** at language-relevant layers and **(ii)** ***budget*** **low-rank capacity** where each language deviates most from a shared multilingual backbone.

We operationalize language channels via a ***depth fingerprint*** derived from **spectral statistics** of layerwise activations. For each language $\lambda \in \{1, \ldots, m\}$ and layer $\ell \in \{0, \ldots, L-1\}$, we form a centered activation matrix $H_{\lambda,\ell} \in \mathbb{R}^{N \times d}$ from hidden states over a fixed prompt pool, compute singular values $\sigma_1 \geq \cdots \geq \sigma_r$, and fit a ***power-law tail*** $\sigma_k \approx Ck^{-\beta_{\lambda,\ell}}$ over $k \in [k_{\min}, k_{\max}]$ in log–log space. We define the ***power-law exponent*** $\alpha_\lambda(\ell) = 1/\widehat{\beta}_{\lambda,\ell}$, yielding the channel curve $\alpha_\lambda : \ell \mapsto \mathbb{R}_+$. To quantify ***depth-localized language divergence***, we compute a cross-language reference $\mu(\ell) = \frac{1}{m}\sum_\lambda \alpha_\lambda(\ell)$ and scale $\sigma(\ell) = \sqrt{\frac{1}{m}\sum_\lambda(\alpha_\lambda(\ell) - \mu(\ell))^2} + \varepsilon$, and define the ***normalized deviation score*** $\tilde{d}_\lambda(\ell) = \frac{|\alpha_\lambda(\ell) - \mu(\ell)|}{\sigma(\ell)}$. PLoRA selects a sparse ***support set*** of adaptation layers $\mathcal{S}_\lambda = \mathrm{TopK}_\ell\, \tilde{d}_\lambda(\ell) \cup \mathcal{S}_{\mathrm{term}}$, where $\mathcal{S}_{\mathrm{term}}$ is an optional ***terminal-window prior*** motivated by empirical localization of alignment and decision formation in late transformer blocks.

Given $\mathcal{S}_\lambda$, PLoRA injects ***language-specific LoRA*** updates **only on those layers** and **only when routing to language** λ. For a target weight matrix W_ℓ (e.g., attention projections or MLP matrices) we apply $\Delta W_\ell^{(\lambda)} = B_\ell^{(\lambda)} A_\ell^{(\lambda)}$ with $A_\ell^{(\lambda)} \in \mathbb{R}^{r_{\lambda,\ell} \times d_{\mathrm{in}}}$ and $B_\ell^{(\lambda)} \in \mathbb{R}^{d_{\mathrm{out}} \times r_{\lambda,\ell}}$, so that $W'_\ell = W_\ell + \Delta W_\ell^{(\lambda)}$. Crucially, ranks are ***budgeted rather than uniform***: under a per-language capacity budget R_λ we allocate ranks by channel salience, e.g., with $w_{\lambda,\ell} = (\tilde{d}_\lambda(\ell))^p$ for $p \geq 1$ and $r_{\lambda,\ell} = \mathrm{clip}\left(r_{\min}, r_{\max}, \left\lfloor R_\lambda \frac{w_{\lambda,\ell}}{\sum_{j \in \mathcal{S}_\lambda} w_{\lambda,j}} \right\rceil\right)$. Training updates only routed adapter parameters $\phi_\lambda = \{(A_\ell^{(\lambda)}, B_\ell^{(\lambda)})\}_{\ell \in \mathcal{S}_\lambda}$ via $\min_{\{\phi_\lambda\}} \sum_\lambda \mathbb{E}_{(x,y)\sim\mathcal{D}_\lambda}[\mathcal{L}_{\mathrm{CE}}(f_{\theta,\phi_\lambda}(x), y)]$, with optional ***geometry-aware regularization*** using differentiable spectral proxies (e.g., stable-rank matching) to control representational concentration without backpropagating through expensive SVD-based tail fits.

We propose an evaluation protocol spanning multilingual translation and instruction following with **(i) per-language quality gains at fixed adapter budgets**, **(ii) interference analysis under mixed-language training**, **(iii) robustness to routing noise and code-switching** via soft mixtures $\Delta W_\ell(x) = \sum_\lambda p(\lambda \mid x) \Delta W_\ell^{(\lambda)}$, and **(iv) geometry diagnostics** verifying that adaptation remains ***depth-localized and channel-consistent*** (before/after $\alpha_\lambda(\ell)$ curves, layer-selection heatmaps, and rank maps). By aligning PEFT capacity with intrinsic ***depth-wise language geometry***, **PLoRA** aims to deliver **stronger minority-language improvements**, **lower cross-language leakage**, and a ***principled, interpretable*** bridge between multilingual representation structure and adapter placement.

1 Polyglotic LoRA (PLoRA): Channel-Aware, Language-Routed Low-Rank Adaptation

Goal. We seek a ***language-routed*** parameter-efficient fine-tuning (PEFT) mechanism that (i) allocates **adapter capacity** to **language-specific depth regions** where internal representations measurably

diverge across languages, and (ii) reduces ***negative transfer*** induced by a single shared low-rank update. **Polyglotic LoRA (PLoRA)** achieves this by ***measuring language channels*** (depth fingerprints) and then using them to ***place*** and ***budget*** LoRA adapters. **We do not require code-mixing support**; routing is *single-language* per input.

1.1 Notation and Setup

Let $\mathcal{L} = \{1, \ldots, m\}$ be the set of languages and let the frozen backbone be a transformer with L layers indexed by $\ell \in \{0, \ldots, L-1\}$. For an input prompt x, denote the hidden state at layer ℓ by $h_\ell(x) \in \mathbb{R}^d$. Backbone parameters are θ; per-language adapter parameters are ϕ_λ for $\lambda \in \mathcal{L}$. Each language has a dataset $\mathcal{D}_\lambda$ (translation, instruction tuning, QA, etc.) and a probe prompt pool $\mathcal{P}_\lambda$ for channel measurement.

LoRA injection (per target matrix). For any target weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA injects

$$W' \;=\; W + \Delta W, \qquad \Delta W \;=\; BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ with $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The backbone W is frozen; only (A, B) are trained.

PLoRA routing (single-language). PLoRA maintains **distinct LoRA parameters per language** and activates **exactly one** adapter set per input. If language is known (dataset label, user selection), routing is deterministic. Otherwise, a lightweight language ID predictor outputs $\hat{\lambda}(x)$ and PLoRA activates $\phi_{\hat{\lambda}(x)}$.

1.2 Step 1: Measuring Language Channels via Spectral Tail Exponents

PLoRA's key primitive is a ***channel curve*** $\alpha_\lambda(\ell)$ capturing language-specific depth geometry.

1.2.1 Step 1.1: Prompt pools and activation matrices

Fix a probe prompt pool $\mathcal{P}_\lambda = \{x_i\}_{i=1}^N$ for each language λ, with equal N across languages and deterministic inference (fixed seeds, same decoding mode). Run the *frozen* backbone and collect layer activations:

$$H_{\lambda,\ell}[i,:] \;=\; h_\ell(x_i) \;-\; \frac{1}{N}\sum_{j=1}^{N} h_\ell(x_j), \qquad H_{\lambda,\ell} \in \mathbb{R}^{N \times d}.$$

Centering removes prompt-set mean, focusing on variability structure.

1.2.2 Step 1.2: Singular spectrum and tail fitting

Compute the singular values of $H_{\lambda,\ell}$:

$$H_{\lambda,\ell} \;=\; U \Sigma V^\top, \qquad \Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r), \qquad \sigma_1 \geq \ldots \geq \sigma_r,$$

where $r = \min(N, d)$. Fit a power-law on the tail:

$$\sigma_k \;\approx\; C\, k^{-\beta_{\lambda,\ell}} \quad \Longleftrightarrow \quad \log \sigma_k \;\approx\; \log C - \beta_{\lambda,\ell} \log k,$$

over $k \in [k_{\min}, k_{\max}]$ (tail window). Estimate $\widehat{\beta}_{\lambda,\ell}$ by least squares and define the ***spectral tail exponent***

$$\alpha_\lambda(\ell) \;:=\; \frac{1}{\widehat{\beta}_{\lambda,\ell}}.$$

This produces a language-specific depth fingerprint $\ell \mapsto \alpha_\lambda(\ell)$.

1.2.3 Step 1.3: Stability via subsampling

To reduce tail-fit noise, estimate across S subsamples $\mathcal{P}_\lambda^{(s)}$ of size N' and report

$$\bar{\alpha}_\lambda(\ell) \;=\; \frac{1}{S}\sum_{s=1}^{S} \alpha_\lambda^{(s)}(\ell), \qquad \mathrm{sd}_\lambda(\ell) \;=\; \sqrt{\frac{1}{S-1}\sum_{s=1}^{S}\Big(\alpha_\lambda^{(s)}(\ell) - \bar{\alpha}_\lambda(\ell)\Big)^2}.$$

In practice, you should only trust deviations that exceed sampling noise.

1.3 Step 2: Channel Divergence Scores Across Depth

We transform channel curves into comparable deviation maps that drive placement and budgeting.

1.3.1 Step 2.1: Cross-language reference

For each layer ℓ, compute mean and dispersion across languages:

$$\mu(\ell) \;=\; \frac{1}{m}\sum_{\lambda\in\mathcal{L}} \bar{\alpha}_\lambda(\ell), \qquad \sigma(\ell) \;=\; \sqrt{\frac{1}{m}\sum_{\lambda\in\mathcal{L}} \big(\bar{\alpha}_\lambda(\ell)-\mu(\ell)\big)^2} + \varepsilon.$$

1.3.2 Step 2.2: Normalized deviation (channel salience)

Define the ***normalized channel deviation***:

$$\tilde{d}_\lambda(\ell) \;=\; \frac{|\bar{\alpha}_\lambda(\ell)-\mu(\ell)|}{\sigma(\ell)}.$$

$\tilde{d}_\lambda(\ell)$ is a per-language importance map over depth: larger values indicate layers where language λ departs most from the shared multilingual spine.

1.4 Step 3: Adapter Placement by Sparse Support Sets

PLoRA chooses a **sparse** set of layers $\mathcal{S}_\lambda$ per language.

1.4.1 Step 3.1: Top-K layer selection

Choose an adaptation budget K_α (e.g., $K_\alpha \in [6, 12]$). Let $\ell_{(1)}, \ldots, \ell_{(L)}$ be layers sorted by $\tilde{d}_\lambda(\ell)$ descending. Define

$$\mathcal{S}_\lambda^{(\alpha)} \;=\; \{\ell_{(1)}, \ldots, \ell_{(K_\alpha)}\}.$$

1.4.2 Step 3.2: Optional terminal-window prior

Many decision-critical transformations concentrate late in depth. To hedge against missing late-layer structure, optionally include

$$\mathcal{S}_{\text{term}} \;=\; \{L-w,\, L-w+1, \ldots, L-1\},$$

for a small w (e.g., $w = 2$ or $w = \lceil 0.1L \rceil$). The final support is

$$\mathcal{S}_\lambda \;=\; \mathcal{S}_\lambda^{(\alpha)} \cup \mathcal{S}_{\text{term}}.$$

1.4.3 Step 3.3: Selection stability (margin condition)

Let $\tilde{d}_\lambda(\ell)$ be the true score and $\widehat{\tilde{d}}_\lambda(\ell) = \tilde{d}_\lambda(\ell) + e_\ell$ an estimate with $|e_\ell| \leq \epsilon$ (use empirical bounds from $\mathrm{sd}_\lambda(\ell)$ propagated through the normalization). If

$$\tilde{d}_\lambda(\ell_{(K_\alpha)}) - \tilde{d}_\lambda(\ell_{(K_\alpha+1)}) \;>\; 2\epsilon,$$

then Top-K_α selection is invariant under estimation noise. If the margin is small, smooth $\bar{\alpha}_\lambda(\ell)$ across depth, reduce K_α, or take a union over subsampling runs.

1.5 Step 4: Rank Budgeting by Channel Salience

PLoRA allocates **rank** per language-layer under a strict parameter budget.

1.5.1 Step 4.1: Parameter accounting

For a LoRA module on matrix W with $d_{\text{in}}, d_{\text{out}}$ and rank r, the parameter cost is

$$\#\text{params}(\Delta W) \;=\; r(d_{\text{in}} + d_{\text{out}}).$$

If a layer adapts multiple matrices (e.g., attention and MLP), sum across targets.

1.5.2 Step 4.2: Budgeted rank allocation

Fix per-language capacity R_λ (in rank-mass or directly in parameter count). For $\ell \in \mathcal{S}_\lambda$, define weights

$$w_{\lambda,\ell} \;=\; \Big(\tilde{d}_\lambda(\ell)\Big)^p, \qquad p \geq 1,$$

and allocate

$$r_{\lambda,\ell} \;=\; \text{clip}\left(r_{\min},\, r_{\max},\, \left\lfloor R_\lambda \frac{w_{\lambda,\ell}}{\sum_{j\in\mathcal{S}_\lambda} w_{\lambda,j}} \right\rfloor\right).$$

This concentrates rank where the language deviates most strongly from the multilingual reference.

Optional fairness across languages. To avoid over-allocating to high-resource languages, set R_λ inversely proportional to resource size, e.g.,

$$R_\lambda \;\propto\; |\mathcal{D}_\lambda|^{-\gamma}, \qquad \gamma \in [0,1],$$

or enforce equal parameter budgets across languages.

1.6 Step 5: Language-Routed Training Objective

PLoRA trains each language adapter on its own data while freezing the backbone.

1.6.1 Step 5.1: Per-language likelihood training

Define the language-routed model $f_{\theta,\phi_\lambda}(\cdot)$ that activates only adapters for language λ. Optimize

$$\min_{\{\phi_\lambda\}_{\lambda\in\mathcal{L}}} \sum_{\lambda\in\mathcal{L}} \mathbb{E}_{(x,y)\sim\mathcal{D}_\lambda}\left[\mathcal{L}_{\mathrm{CE}}(f_{\theta,\phi_\lambda}(x), y)\right].$$

Implementation: for each minibatch, group examples by language and run independent forward/backward passes so that gradients update only the corresponding ϕ_λ (avoid cross-language moment leakage in optimizer states by keeping optimizer states per adapter).

1.6.2 Step 5.2: Optional geometry-aware regularization (differentiable proxy)

Directly regularizing $\alpha_\lambda(\ell)$ is costly. If geometry control is needed, use a differentiable proxy such as stable rank:

$$\mathrm{sr}(H) \;=\; \frac{\|H\|_F^2}{\|H\|_2^2},$$

where $\|H\|_2$ can be approximated by a few power iterations. Let $\mathrm{sr}^\star_{\lambda,\ell}$ be a target. Define

$$\mathcal{R}_{\mathrm{chan}}(\lambda) \;=\; \sum_{\ell\in\mathcal{S}_\lambda} \left(\log \mathrm{sr}(H_{\lambda,\ell}) - \log \mathrm{sr}^\star_{\lambda,\ell}\right)^2,$$

and train with

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{CE}} + \lambda_{\mathrm{chan}} \sum_{\lambda\in\mathcal{L}} \mathcal{R}_{\mathrm{chan}}(\lambda).$$

This constrains spectral concentration without differentiating through SVD tail fits.

1.7 Step 6: Practical Implementation Checklist (Exact Steps)

1. **Fix languages and tasks.** Freeze tokenizer; fix decoding settings; prepare per-language train/dev/test.
2. **Build probe pools $\mathcal{P}_\lambda$.** Equal N; deterministic inference; store prompts for reproducibility.
3. **Extract activations.** For each λ, ℓ, compute $H_{\lambda,\ell}$ from hidden states and center across prompts.
4. **Compute channel curves.** SVD, tail fit, $\alpha_\lambda(\ell)$, and subsampling stability statistics.
5. **Compute deviation maps.** $\mu(\ell), \sigma(\ell)$ and $\tilde{d}_\lambda(\ell)$ for all λ, ℓ.
6. **Select supports.** Top-K_α layers per language; optionally add the terminal window; verify selection margins.
7. **Allocate ranks.** Choose $(r_{\min}, r_{\max})$ and budgets R_λ; compute $r_{\lambda,\ell}$; verify parameter totals.
8. **Instantiate adapters.** Attach LoRA modules only at $\ell \in \mathcal{S}_\lambda$ for each language λ.
9. **Train routed adapters.** Optimize per-language likelihood (plus optional geometry regularizer); freeze θ.
10. **Evaluate.** Compare to shared LoRA, per-language LoRA with uniform layers, and adaptive-rank baselines; report quality, interference, and geometry plots (before/after $\alpha_\lambda(\ell)$).

1.8 What to Claim (and What to Show)

Claim 1 (capacity alignment). PLoRA aligns PEFT capacity with intrinsic depth-wise language geometry: layers with large $\tilde{d}_\lambda(\ell)$ receive both **adapter placement** and **higher rank**.

Claim 2 (reduced interference). Language routing prevents gradient mixing across incompatible channels, reducing negative transfer relative to shared adapters.

Claim 3 (interpretability). Support sets $\mathcal{S}_\lambda$ and rank maps $r_{\lambda,\ell}$ are interpretable artifacts that expose *where* and *how much* the model adapts per language.

Minimal figures. (1) Heatmap of $\tilde{d}_\lambda(\ell)$ with $\mathcal{S}_\lambda$ overlays; (2) rank map $r_{\lambda,\ell}$; (3) before/after $\alpha_\lambda(\ell)$ curves; (4) per-language gains vs parameter budget.

2 Datasets and Experimental Plan

Overview. We evaluate **Polyglotic LoRA (PLoRA)** as a ***channel-aware, language-routed*** PEFT method under **single-language routing** (each example is assigned exactly one language ID; **no code-mixing**). Our experimental plan targets three questions: **(Q1)** does channel-aware *placement* and *rank budgeting* improve per-language quality at fixed adapter budgets; **(Q2)** does language routing reduce cross-language interference relative to shared adapters; **(Q3)** do the learned updates respect *language-channel locality* (geometry diagnostics).

2.1 Languages

We focus on the same $m = 12$ languages used in our channel analysis: **Hindi, French, Chinese, Urdu, English, Awadhi, Bengali, Marathi, Dutch, Polish, Sindhi, and South Azerbaijani**. Each training and evaluation example carries a single language label $\lambda \in \mathcal{L}$ obtained from dataset metadata or deterministic preprocessing.

2.2 Benchmarks and Datasets

Machine Translation (MT). We adopt MT as our primary benchmark because it provides a clean, language-separated objective and enables controlled measurement of **negative transfer**. For evaluation, we use **FLORES-200** (NLLB Team et al., 2022) (dev / devtest where available) as the standard multilingual test suite. FLORES-200 is distributed through the NLLB release and widely used for evaluating translation across $200+$ languages and tens of thousands of directions (NLLB Team et al., 2022).

For training, we construct per-language parallel corpora $\{\mathcal{D}_\lambda^{\text{MT}}\}$ from **OPUS** (Tiedemann, 2012) (and optionally additional curated corpora if available). We consider two training regimes:

- **English-centric MT:** for each $\lambda \neq$ English, train on $\lambda \leftrightarrow$ English bitext.
- **Many-to-English MT:** for each $\lambda \neq$ English, train only on $\lambda \rightarrow$ English, yielding a more controlled comparison across languages.

Multilingual Instruction Tuning (IT). To test general-purpose multilingual instruction following, we use **xP3** (Muennighoff et al., 2022), a large multitask dataset spanning many languages with English and machine-translated prompts, and widely used in multilingual instruction-following and multitask finetuning settings. We form per-language instruction subsets $\{\mathcal{D}_\lambda^{\text{IT}}\}$ by filtering xP3 by language metadata and ensuring each example is assigned exactly one language ID (Muennighoff et al., 2022).

2.3 Evaluation Metrics

For MT, we report both a reproducible surface-form metric and a learned metric correlated with human judgments:

- **SacreBLEU** (Post, 2018) for standardized BLEU reporting and comparability across labs.
- **COMET** (Rei et al., 2020) as a neural MT evaluation framework with strong correlation to human evaluation.

We report per-language scores, macro averages across languages, and a worst-k average (e.g., worst-5 languages) to highlight low-resource / minority-language behavior.

2.4 Baselines

We compare PLoRA against the following baselines under **matched total adapter parameter budget**:

- **Shared LoRA (S-LoRA):** one adapter shared across all languages (maximal sharing, maximal interference risk).
- **Per-language LoRA (Uniform-Place):** separate adapter per language placed on a fixed language-agnostic layer set (e.g., last w layers) with uniform rank.
- **Per-language LoRA (Random-Place):** separate adapter per language placed on $|\mathcal{S}_\lambda|$ randomly selected layers (capacity matched, no channel guidance).
- **Adaptive-rank baseline:** a global adaptive-rank method (e.g., AdaLoRA-style) without language-channel placement or language-routed sparsity.

Where feasible, we additionally include **full fine-tuning** on a smaller backbone or reduced compute budget as an upper bound.

2.5 PLoRA Variants and Ablations

We run a structured ablation grid isolating channel measurement, placement, and budgeting:

- **PLoRA-α (main):** Top-K_α layers per language by normalized deviation $\tilde{d}_\lambda(\ell)$.
- **PLoRA-Terminal:** adapters only on the last w layers (tests "late-only" hypothesis).
- **PLoRA-Random:** $|\mathcal{S}_\lambda|$ random layers per language (tests whether sparsity alone explains gains).
- **PLoRA-Union:** all languages share the union support $\cup_\lambda \mathcal{S}_\lambda$ (tests routing vs. placement).
- **Fixed-rank vs Channel-budgeted ranks:** constant rank on selected layers vs rank allocation proportional to $(\tilde{d}_\lambda(\ell))^p$.

2.6 Training Protocol

Language-routed batching. We train using mixed-language minibatches but **partition each minibatch by language** and perform independent forward/backward passes per language group, ensuring gradients update only the corresponding adapter parameters ϕ_λ. Optimizer states are maintained per language adapter to prevent cross-language moment leakage.

Budget matching. We match total trainable adapter parameters between S-LoRA and PLoRA. When PLoRA is sparser in depth, we either increase rank on selected layers to match the budget or keep rank fixed and report the effective parameter reduction as an efficiency gain.

2.7 Negative Transfer Measurement

To quantify interference, we compare multilingual training to language-only training under matched budgets. For each language λ, define:

$$\Delta_\lambda \;=\; \mathrm{Score}_\lambda(\text{Multilingual}) \;-\; \mathrm{Score}_\lambda(\text{Language-only}).$$

We report the distribution of Δ_λ (mean, median, worst-case) and expect PLoRA to reduce negative transfer relative to shared adapters.

2.8 Geometry and Channel Diagnostics

PLoRA is designed to be **interpretable** through explicit placement and budgeting decisions. We therefore include:

- **Deviation heatmaps:** visualize $\tilde{d}_\lambda(\ell)$ with supports $\mathcal{S}_\lambda$ overlaid.
- **Rank maps:** visualize $r_{\lambda,\ell}$ as a sparse language-by-layer allocation matrix.
- **Before/after channel curves:** plot $\alpha_\lambda(\ell)$ before and after adaptation to verify depth-localized, channel-consistent change.
- **Channel–gain correlation:** correlate aggregate salience $\sum_{\ell \in \mathcal{S}_\lambda} \tilde{d}_\lambda(\ell)$ with per-language gains.

2.9 Minimal Three-Experiment Plan

Experiment 1 (Main MT). Train S-LoRA vs PLoRA under matched budgets on OPUS-derived bitext and evaluate on FLORES-200; report SacreBLEU and COMET per language and macro averages (Tiedemann, 2012; NLLB Team et al., 2022; Post, 2018; Rei et al., 2020).

Experiment 2 (Ablations). Run placement ablations (PLoRA-α vs Terminal vs Random vs Union) and budgeting ablations (Fixed-rank vs Channel-budgeted) to isolate which components drive gains.

Experiment 3 (Geometry validation). Report deviation heatmaps, rank maps, and before/after $\alpha_\lambda(\ell)$ curves to validate the channel hypothesis.

References

Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M. Saiful Bari, Sheng Shen, Zheng-Xin Yong, Hailey Schoelkopf, et al. 2022. Crosslingual generalization through multitask finetuning. *arXiv preprint arXiv:2211.01786*.

NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, et al. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.

Matt Post. 2018. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 186–191. Association for Computational Linguistics.

Ricardo Rei, Craig Stewart, Ana C. Farinha, and Alon Lavie. 2020. COMET: A neural framework for MT evaluation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2685–2702. Association for Computational Linguistics.

Jörg Tiedemann. 2012. Parallel data, tools and interfaces in OPUS. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 2214–2218, Istanbul, Turkey. European Language Resources Association (ELRA).

Algorithm 1 Polyglotic LoRA (PLoRA): Channel-Aware, Language-Routed Low-Rank Adaptation

Require: Languages $\mathcal{L} = \{1, \ldots, m\}$; frozen transformer f_θ with layers $\ell \in \{0, \ldots, L-1\}$ and hidden size d
Require: Per-language training sets $\{\mathcal{D}_\lambda\}_{\lambda \in \mathcal{L}}$
Require: Per-language probe pools $\{\mathcal{P}_\lambda\}_{\lambda \in \mathcal{L}}$ with $|\mathcal{P}_\lambda| = N$
Require: Target matrices per layer $\mathcal{M}$ (e.g., $\{W_q, W_v, W_o, W_{\text{up}}, W_{\text{down}}\}$)
Require: Tail window indices $k_{\min}, k_{\max}$; subsampling rounds S; subsample size N'
Require: Support budget K_α; terminal window size w (set $w = 0$ to disable)
Require: Rank bounds $r_{\min}, r_{\max}$; per-language rank budgets $\{R_\lambda\}$; exponent $p \geq 1$
Require: Training steps T; batch size B; optimizer Opt; optional $\lambda_{\text{chan}} \geq 0$
Ensure: Language adapters $\{\phi_\lambda\}$, supports $\{\mathcal{S}_\lambda\}$, ranks $\{r_{\lambda,\ell}\}$, channel curves $\{\bar{\alpha}_\lambda(\ell)\}$

Phase I: Measure language channels $\bar{\alpha}_\lambda(\ell)$
for $\lambda \in \mathcal{L}$ **do**
  for $\ell \leftarrow 0$ to $L-1$ **do**
    Initialize list $\mathcal{A} \leftarrow [\,]$
    for $s \leftarrow 1$ to S **do**
      Sample $\mathcal{P}_\lambda^{(s)} \subset \mathcal{P}_\lambda$ with $|\mathcal{P}_\lambda^{(s)}| = N'$
      Collect hidden states $\{h_\ell(x) \in \mathbb{R}^d : x \in \mathcal{P}_\lambda^{(s)}\}$ using frozen f_θ
      Form centered activation matrix $H_{\lambda,\ell}^{(s)} \in \mathbb{R}^{N' \times d}$ by row-centering across prompts
      Compute singular values $\sigma_1 \geq \cdots \geq \sigma_r$ of $H_{\lambda,\ell}^{(s)}$ where $r = \min(N', d)$
      Fit tail in log–log space over $k \in [k_{\min}, k_{\max}]$:
        $\log \sigma_k \approx c - \beta_{\lambda,\ell}^{(s)} \log k$ (least squares)
      Set $\alpha_\lambda^{(s)}(\ell) \leftarrow 1/\beta_{\lambda,\ell}^{(s)}$ and append to $\mathcal{A}$
    end for
    Set $\bar{\alpha}_\lambda(\ell) \leftarrow \mathrm{mean}(\mathcal{A})$ and $\mathrm{sd}_\lambda(\ell) \leftarrow \mathrm{stdev}(\mathcal{A})$
  end for
end for
Phase II: Build cross-language reference and deviation maps $\tilde{d}_\lambda(\ell)$
for $\ell \leftarrow 0$ to $L-1$ **do**
  $\mu(\ell) \leftarrow \frac{1}{m} \sum_{\lambda \in \mathcal{L}} \bar{\alpha}_\lambda(\ell)$
  $\sigma(\ell) \leftarrow \sqrt{\frac{1}{m} \sum_{\lambda \in \mathcal{L}} (\bar{\alpha}_\lambda(\ell) - \mu(\ell))^2} + \varepsilon$
end for
for $\lambda \in \mathcal{L}$ **do**
  for $\ell \leftarrow 0$ to $L-1$ **do**
    $\tilde{d}_\lambda(\ell) \leftarrow |\bar{\alpha}_\lambda(\ell) - \mu(\ell)| / \sigma(\ell)$
  end for
end for
Phase III: Select supports $\mathcal{S}_\lambda$ **and allocate ranks** $r_{\lambda,\ell}$
for $\lambda \in \mathcal{L}$ **do**
  $\mathcal{S}_\lambda^{(\alpha)} \leftarrow$ indices of Top-K_α layers by $\tilde{d}_\lambda(\ell)$
  if $w > 0$ **then**
    $\mathcal{S}_{\text{term}} \leftarrow \{L-w,\ L-w+1,\ \ldots,\ L-1\}$
  else
    $\mathcal{S}_{\text{term}} \leftarrow \emptyset$
  end if
  $\mathcal{S}_\lambda \leftarrow \mathcal{S}_\lambda^{(\alpha)} \cup \mathcal{S}_{\text{term}}$
  for all $\ell \in \mathcal{S}_\lambda$ **do**
    $u_{\lambda,\ell} \leftarrow (\tilde{d}_\lambda(\ell))^p$
  end for
  $Z_\lambda \leftarrow \sum_{j \in \mathcal{S}_\lambda} u_{\lambda,j}$
  for all $\ell \in \mathcal{S}_\lambda$ **do**
    $r_{\lambda,\ell} \leftarrow \mathrm{round}\Big(R_\lambda \cdot \frac{u_{\lambda,\ell}}{Z_\lambda}\Big)$
    $r_{\lambda,\ell} \leftarrow \min\{r_{\max}, \max\{r_{\min}, r_{\lambda,\ell}\}\}$
  end for
  Initialize adapter container $\phi_\lambda \leftarrow \emptyset$
  for all $\ell \in \mathcal{S}_\lambda$ **do**

3 Appendix